

Part A) Solution

Question 1) Short Question Answers

- 1) Which of the following grammars can be parsed by LR(0) but not by SLR(1), Give one line reason also?
- A) $S \rightarrow a A, A \rightarrow b$
 - B) $S \rightarrow A a \mid b, A \rightarrow c$
 - C) $S \rightarrow a A a, A \rightarrow b$

Solution : **None**

- 2) Which of the following grammars will cause a shift-reduce conflict in an LR(0) parser, Give reason ?
- A) $S \rightarrow a A, A \rightarrow b$
 - B) $S \rightarrow A a \mid b, A \rightarrow c$
 - C) $S \rightarrow a A b, A \rightarrow c$
 - D) $S \rightarrow A, A \rightarrow a A \mid \epsilon$

Solution: **D) $S \rightarrow A, A \rightarrow a A \mid \epsilon$**

Reason: In an LR(0) parser, a shift-reduce conflict occurs because the grammar's ϵ -production ($A \rightarrow \epsilon$) allows a reduction in the same state where a terminal ('a') can be shifted, creating ambiguity in the parsing decision.

- 3) For the grammar: $S \rightarrow A a \mid B, A \rightarrow b B, B \rightarrow c \mid \epsilon$. What is the DFA state that will exhibit a shift-reduce conflict?

Solution: Start state or state with **$[A \rightarrow b . B], [B \rightarrow . c], [B \rightarrow .]$**

- 4) Consider the grammar:

$S \rightarrow A B, A \rightarrow a \mid \epsilon, B \rightarrow b \mid \epsilon$. With the augmented rule $S' \rightarrow S$, how many states are in the LR(0) DFA, accounting for all closures and transitions?

The distinct states are:

1. State 0: $S' \rightarrow .S, S \rightarrow .AB, A \rightarrow .a, A \rightarrow .\epsilon$
2. State 1: $S' \rightarrow S.$
3. State 2: $S \rightarrow A.B, B \rightarrow .b, B \rightarrow .\epsilon$
4. State 3: $A \rightarrow a.$
5. State 4: $S \rightarrow AB.$
6. State 5: $B \rightarrow b.$

Solution :

Total: **6 states.**

5) Consider the Grammar

$S \rightarrow A a \mid B a$
 $A \rightarrow c A \mid d$
 $B \rightarrow c B \mid d$

In SLR(1), a reduce-reduce conflict occurs in a state with $[A \rightarrow d.]$ and $[B \rightarrow d.]$. Why does this conflict persist despite follow sets?

Solution : **FOLLOW(A) and FOLLOW(B) both include a, causing ambiguity.**

6) Consider the following grammar: Is it LR(0) or SLR(1) or Both give reason?

$S \rightarrow aAc \mid aBd$
 $A \rightarrow b$
 $B \rightarrow b$

Solution: **Not LR(0):** The grammar has a reduce-reduce conflict in State 5 ($A \rightarrow b A \rightarrow b A \rightarrow b A$ vs. $B \rightarrow b B \rightarrow b B \rightarrow b B$) in the LR(0) parser, as it cannot decide between the reductions without lookahead.

SLR(1): The conflict is resolved using FOLLOW sets ($\text{FOLLOW}(A) = \{c\}$, $\text{FOLLOW}(A) = \{c\}$, $\text{FOLLOW}(A) = \{c\}$, $\text{FOLLOW}(B) = \{d\}$, $\text{FOLLOW}(B) = \{d\}$, $\text{FOLLOW}(B) = \{d\}$), which are disjoint, making the grammar unambiguous with one-symbol lookahead.

Answer: The grammar is **SLR(1) but not LR(0)**.

Question no 2)

1) Consider the following Grammar

$S \rightarrow A B C \mid D a$
 $A \rightarrow b A c \mid B d \mid \epsilon$
 $B \rightarrow e B f \mid A C \mid \epsilon$
 $C \rightarrow g \mid \epsilon$
 $D \rightarrow h D \mid S C b$

- a) Compute the First sets for all non-terminals (S, A, B, C, D). [5 marks]
 b) Compute the Follow sets for all non-terminals (S, A, B, C, D). [5 marks]

	First Set	Follow Set
S	beghd	\$bg
A	begd epsilon	\$begdf
B	begh epsilon	\$bdgf
C	g epsilon	\$ bdfg
D	hbegd	a

- a) Determine if this grammar is LL(1). Justify your answer by analyzing potential conflicts. [5 marks]

Conflict: {b, e, g} overlaps between productions. The grammar is **not LL(1)** due to overlapping First sets for S's productions. Or indirect left recursion

Question no 3)

2) Consider the following Grammar

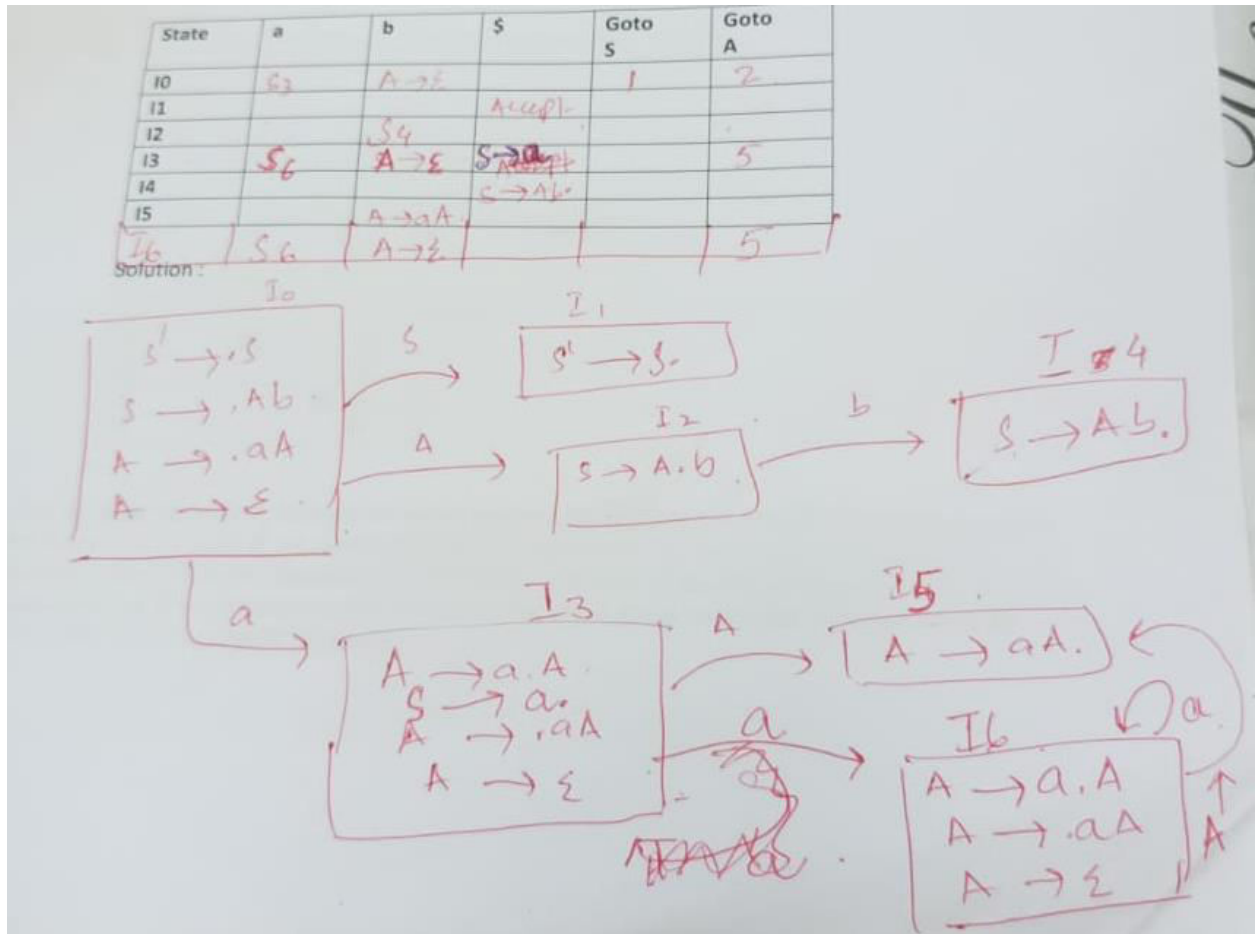
$S' \rightarrow S$
 $S \rightarrow A b \mid a$
 $A \rightarrow a A \mid \epsilon$

Construct the DFA and SLR (1) table for the above grammar

State	a	b	\$	Goto S	Goto A
10					
11					

12					
13					
14					
15					

Solution :



Question no 4)

4) Consider the following context-free grammar:

$$S \rightarrow aSb \mid ab$$

Following is the SLR(1) Parsing table for the Grammar, where is

R1 = Reduce $S \rightarrow aSb$

R2 = $S \rightarrow ab$

State	a	b	\$	Goto S
I0	S2			1
I1			Accept	
I2	S2	S4		3
I3		S5		
I4		R2	R2	
I5		R1	R1	

You are using an SLR(1) bottom-up parser to parse the input string aaabb. The current state of the parser's stack is [a, S, b], where the stack is represented by the sequence of symbols (ignoring state numbers). Based on this grammar, the SLR(1) parsing mechanism, and the given stack configuration, determine what portion of the input string aaabb has been parsed so far and what remains to be parsed? **[5 marks]**

Fill the following table

Step	Stack	Input	Lookahead	Action
1	[0]	aaabb\$		
2				
3				
4				
5				
6				
7				

Solution :

The stack `[a, S, b]` as the exact symbol sequence doesn't appear cleanly because SLR(1) retains prior context (e.g., `[0, a, 2, ...]`). The question likely intends the top of the stack to be `[a, S, b]`. Let's assume a minimal stack:

Step	Stack	Input	Lookahead	Action
1	[0]	aaabb\$	a	s2
2	[0, a, 2]	aabb\$	a	s2
3	[0, a, 2, a, 2]	abb\$	a	s2
4	[0, a, 2, a, 2, a, 2]	bb\$	b	s4
5	[0, a, 2, a, 2, a, 2, b, 4]	b\$	b	r2 ($S \rightarrow ab$)
6	[0, a, 2, a, 2, S, 3]	b\$	b	s5

- After step 6: `[0, a, 2, a, 2, S, 3]`, shift `b`:
- Step 7: `[0, a, 2, a, 2, S, 3, b, 5]`, top symbols `[a, S, b]` (from `[a, 2]`, `S`, `b`).

